

Data Quality Findings Report

Phase 1: Property Valuation & Market Intelligence

Author: **Bright Uzosike** — Data Scientist

Executive Summary

This report documents the data-quality assessment performed on the AlloyTower property dataset (*alloy_data.csv*, 2,090 rows, 28 columns) prior to building the Phase 1 Automated Valuation Model. The assessment combined structural checks, missingness analysis, value-range plausibility, cross-field consistency, and feature-target signal analysis.

The headline finding is that the dataset is **partially synthetic**: the core transactional fields (sale price, property attributes, assessments) are real and internally consistent, but several contextual fields (geographic identifiers, sale dates) appear to have been randomly generated. This shapes both the modelling approach and the realistic accuracy expectations for Phase 1.

The full set of findings is summarised below, then detailed individually in the rest of this report.

Findings at a Glance

#	Finding	Severity	Decision
1	BOM and delimiter issues at file load	LOW	Resolved during reload
2	Redundant primary key (property_id and id are 1-to-1)	LOW	Drop id, retain property_id
3	Structural missingness in unit column (49.7% NaN)	INFO	Leave as-is; not a true gap
4	Latitude and longitude are randomly distributed	CRITICAL	Drop both columns
5	Zip codes are randomly generated	CRITICAL	Drop column
6	Implausible price/size pairings at low end (6.4%)	HIGH	Drop 134 rows below \$50/sqft
7	Sale prices show no real temporal trend	HIGH	Drop date-based features
8	Physical attributes carry weak predictive signal	HIGH	Adjust accuracy expectations; build two models

#	Finding	Severity	Decision
9	State alone explains 57% of price variance	INFO	Use state and city as primary features
10	assessed_value correlates 0.99 with sale price	MEDIUM	Build two models (with/without)
11	Derived/leakage columns identified	MEDIUM	Drop price_per_sqft, annual_tax, building_age

Severity Legend

- **CRITICAL:** the column or value cannot be used at all and must be dropped or replaced; would cause silent failures in production.
- **HIGH:** materially affects modelling decisions or expected performance; requires explicit handling.
- **MEDIUM:** design decision required, but the data is usable in either direction.
- **LOW:** noted but easily resolved; minor cleanup.
- **INFO:** informational; not a defect but worth recording for transparency.

Dataset Overview

Source file	alloy_data.csv (delimited by semicolon, UTF-8 with BOM)
Rows (raw)	2,090
Rows (after cleaning)	1,956 (93.6% retained)
Columns	28 raw; 7-9 retained for modelling
Date range	2016-04-18 to 2026-04-11 (sale dates)
Geographic coverage	84 cities, 80 counties, multiple US states
Property types	Single Family (40%), Condo (30%), Townhouse (20%), Multi Family (10%)
Target variable	last_sale_price (range \$121K–\$3.5M, median \$611K)

Detailed Findings

1. BOM and delimiter issues at file load

LOW

Observation: The CSV file uses a semicolon (;) delimiter rather than the conventional comma, and contains a UTF-8 byte-order-mark (BOM) at the start. A naive `pd.read_csv()` call therefore failed with a `ParserError`, and a subsequent successful load left the BOM character (`\uffeff`) prefixed to the first column name.

Impact: Without correct loading, the dataset cannot be parsed at all, and column lookups by name silently fail when the BOM is present.

Resolution: Reload with `sep=";"`, `encoding="utf-8-sig"` (which strips the BOM), `thousands=","` for numeric columns, and explicit dtype hints. Defensive column-name cleaning (`str.replace("\uffeff", "")`) was added as a fallback.

2. Redundant primary key (property_id and id are 1-to-1)

LOW

Observation: Both `property_id` (formatted as 'P-100001') and `id` (an integer in the 10001–12226 range) are unique across all 2,090 rows, with zero duplicates in either column. Cross-checking confirmed they are perfectly 1-to-1: each `id` corresponds to exactly one `property_id` and vice versa.

Impact: Two identifier columns convey the same information. Including both adds noise and slightly inflates dimensionality without adding signal.

Decision: Retain `property_id` as the canonical key (it follows AlloyTower's internal naming convention and is more human-readable). Drop `id` from the modelling feature set.

3. Structural missingness in unit column (49.7%)

INFO

Observation: The `unit` column has 1,038 NaN values out of 2,090 (49.7%). Cross-tabulation against `property_type` shows the missingness is fully structural: 100% of Single Family and 100% of Multi Family rows have NaN units, while 100% of Condo and 100% of Townhouse rows have populated units. No missingness is unexplained.

Impact: None on modelling — the information is already encoded in `property_type`. The column itself is a high-cardinality string and is not used as a model feature regardless.

Decision: Leave the column unchanged for analytics and address-display purposes. Drop from the modelling feature set as a high-cardinality identifier.

4. Latitude and longitude are randomly distributed

CRITICAL

Observation: Of 841 rows tested against their stated state's bounding box (using a lookup of US state coordinate ranges), only 26 rows (3.1%) had latitude/longitude pairs falling inside the correct state.

Examples: Honolulu, HI mapped to (27.49, -144.86) — middle of the Pacific Ocean; Pittsburgh, PA mapped to (28.37, -81.37) — central Florida; Bronx, NY mapped to (38.76, -95.94) — central Kansas.

Diagnostic: Two rows in Naperville, IL had completely different coordinate pairs (31.49/-98.56 versus 43.60/-113.03), ruling out any systematic projection error or constant offset. The values appear randomly distributed inside a US bounding box.

Impact: Coordinates cannot be used for any spatial feature engineering, geocoding, or visualisation.

Decision: Drop latitude and longitude from the dataset. Location signal will come from state, city, and county (which were verified clean — see Finding 9).

5. Zip codes are randomly generated

CRITICAL

Observation: Of 2,090 zip codes, 2,064 are unique — a 99% uniqueness rate. All values are valid 5-digit numeric strings, but inspection shows no relationship between zip codes and stated city/state.

Examples: Honolulu, HI was assigned zip 46048 (an Indiana zip code); Pittsburgh, PA was assigned 87236 (an unassigned code in New Mexico's range).

Diagnostic: The same pattern as latitude/longitude — values appear randomly drawn from the 10000–99999 range with no relationship to actual geography.

Impact: Zip codes cannot be used as a location feature. Cannot be cross-referenced with external zip-level data (Census, school ratings, etc.) for feature enrichment.

Decision: Drop zip_code from the dataset. State and city remain as the primary location features.

6. Implausible price/size pairings at the low end

HIGH

Observation: 134 rows (6.4% of the dataset) have a price_per_sqft below \$50, and 307 rows (14.7%) below \$75. Examples include a 6,127 sqft condo in Reno priced at \$125,426 (\$20/sqft) and a 4,660 sqft single family home in Las Vegas priced at \$124,781 (\$27/sqft).

Diagnostic: US residential construction cost alone is approximately \$150–\$300/sqft. Sale prices below \$50/sqft are below construction cost and not economically realistic for standing structures, regardless of market. The issue is distributed across all four property types (4.6%–7.7% of each) and across multiple states.

Impact: These rows would teach the model that 6,000 sqft properties can sell for under \$130K, biasing price predictions downward — particularly damaging for properties in mid-tier markets.

Decision: Drop the 134 rows where price_per_sqft < \$50. Retain rows in the \$50–\$75 band, which may include legitimate distressed or rural sales. After cleaning: 1,956 rows retained (93.6%).

7. Sale prices show no real temporal trend

HIGH

Observation: Median sale price grouped by year (2016–2026) varies within a narrow \$608K–\$670K band — approximately 10% spread across an entire decade. The lowest median in the series is 2022 (\$608K), and the highest is 2023 (\$670K).

Diagnostic: Real US housing prices appreciated approximately 40% over 2016–2026 according to Case-Shiller and FHFA indices, with a peak in 2022 followed by a small correction. The pattern in this dataset

shows neither the appreciation nor the 2022 peak — instead, 2022 is the local minimum. This indicates that sale dates were assigned with weak or no relationship to price.

Impact: Date-based features (`last_sale_date`, `sale_year`, `days_since_sale`) carry no usable predictive signal.

Decision: Drop date-based features from the model. Treat the AVM as a cross-sectional valuation model rather than a time-aware one. No inflation adjustment is needed because the prices do not follow inflation in this dataset.

8. Physical attributes carry weak predictive signal

HIGH

Observation: Pearson correlations of physical attribute columns with `last_sale_price`: `sqft` 0.22, `lot_size_sqft` 0.13, `bedrooms` 0.01, `bathrooms` 0.00, `building_age` 0.01, `year_built` -0.01. Boxplots of price by `property_type` show identical distributions across Single Family, Condo, Townhouse, and Multi Family.

Diagnostic: In real US property data, `sqft` typically correlates 0.6–0.8 with sale price and is the dominant physical predictor. Bedrooms and bathrooms typically show moderate positive correlations (0.3–0.5). Property type typically shows clear price separation between categories. None of these patterns hold here.

Impact: The model cannot learn fine-grained property-level adjustments. Realistic accuracy expectations should be revised: for a model excluding `assessed_value`, R^2 is likely capped around 0.65–0.70 with MAPE around 25–35%, primarily driven by location features alone.

Decision: Include physical attributes in the model regardless — tree-based models will assign them low importance automatically and the inclusion is harmless. But communicate the realistic accuracy ceiling clearly to stakeholders. Recommend onboarding a dataset with verifiable physical attributes for Phase 2.

9. State alone explains 57% of price variance

INFO

Observation: Computing the R^2 of a state-only mean-prediction baseline gives 0.566. That is, 57% of the variance in `last_sale_price` is explained by knowing the property's state. Boxplots show a clear two-tier split: high-tier states (CA, WA, HI, NY, MA) with medians of \$1.3M–\$1.9M, and lower-tier states (OR, MD, CO, VA, ID) with medians of \$0.5M–\$0.9M.

Diagnostic: In real US property datasets, state alone typically explains 20–30% of variance because within-state variation (driven by city, neighbourhood, and property attributes) is also large. The 57% figure here is unusually high and is consistent with the rest of the findings — within-state variation is artificially compressed because physical attributes carry weak signal.

Implication: Location features (state, city, county) will dominate the model. Target encoding by state and city is recommended for compactness and for lift over one-hot encoding.

Decision: Use state, city, and county as primary features. Apply target encoding (mean price per group) fitted on training data and applied to test data.

10. assessed_value correlates 0.99 with sale price

MEDIUM

Observation: The Pearson correlation of `assessed_value` with `last_sale_price` is 0.99. The assessment-to-sale ratio (`assessed_value / last_sale_price`) is tightly distributed: mean 0.95, median 0.95, standard deviation 0.09, range 0.80–1.10. `annual_tax` (which is computed from `assessed_value`) correlates 0.91 with `last_sale_price`.

Diagnostic: In real US assessment data, ratios vary much more widely (typically 0.4–1.2 with sometimes much wider ranges) because jurisdictions assess at different rates and assessment cycles vary. The tight 0.95 ± 0.09 distribution here suggests `assessed_value` was generated as a near-perfect proportional approximation of sale price.

Impact: If `assessed_value` is included as a feature, the model will rely on it almost exclusively and achieve very high accuracy ($R^2 \approx 0.99$, MAPE 5–8%) but will not be learning meaningful physical relationships. The AVM becomes essentially a 'correct the assessment' model rather than a fundamentals-based valuation model.

Decision: Build two models in Phase 1: (A) Assessment-aware AVM that includes `assessed_value` — high accuracy, useful when an assessment exists; (B) Fundamentals-only AVM that excludes `assessed_value`, `annual_tax`, and `price_per_sqft` — lower accuracy but applicable to new builds and unassessed properties. Present both in the final report.

11. Derived and leakage columns identified

MEDIUM

Observation: Several columns are derivable from others: `price_per_sqft` = `last_sale_price / sqft` (verified, zero mismatches); `annual_tax` = a fixed percentage of `assessed_value` (correlation 0.91 with each other); `building_age` + `year_built` = 2024 for every row (perfect collinearity); `days_since_sale` and `sale_year` are perfectly negatively correlated (-1.00).

Impact: Including derived columns alongside their source columns introduces redundancy and, for `price_per_sqft`, direct target leakage.

Decision: Drop `price_per_sqft` (target leakage), `annual_tax` (redundant with `assessed_value`), `building_age` (redundant with `year_built` — drop `building_age` because it is anchored to 2024 and will become stale), `days_since_sale` and `sale_year` (redundant temporal columns with no signal — see Finding 7).

Hypothesis: Partially Synthetic Dataset

Pulling the eleven findings together, a coherent picture emerges about how this dataset was likely constructed.

What appears authentic

- Core property attributes (sqft, bedrooms, bathrooms, year_built, lot size, property type) — present, in plausible ranges, internally consistent.
- State and city assignments — clean, with no city-in-wrong-state issues, and matching expected US distributions (TX, FL, CA, NY at the top).
- Sale prices — distributed in plausible ranges with the expected right skew.
- Owner-occupied flag — split 59% True / 41% False, consistent with national homeownership figures.

What appears synthetic or randomised

- Latitude and longitude — random within a US bounding box.
- Zip codes — random within the 10000–99999 range.
- Sale dates — assigned with no relationship to price (real US prices appreciated 40% over the period; this dataset shows ~10% with no trend).
- Physical attributes vs. price — appear independent, suggesting attributes were assigned without conditioning on price.
- Assessed values — generated as a tight proportional approximation of sale price ($95\% \pm 9\%$).

What this means

We are working with an anonymised dataset where the real or realistic properties were randomised. Values were possibly augmented or generated for teaching purposes. The transactional core was preserved, but the relationships between physical attributes and price were not strongly preserved or were diluted in the generation process.

This characterisation is shared with stakeholders not as a criticism of the dataset, but to set realistic expectations about what the AVM can and cannot learn from it.

Implications for Phase 1 Modelling

Final feature set

Column	Status	Reason
state	Keep	Strong location signal — 57% of variance explained alone
city	Keep	Finer-grained location signal; 84 unique cities, ~25 sales each on average
county	Keep	Mostly redundant with city; let the model decide
property_type	Keep	Weak signal in this dataset, but harmless to include
sqft, lot_size_sqft	Keep	Weak signal but theoretically meaningful; tree models will down-weight
bedrooms, bathrooms	Keep	Near-zero correlation with price; included for completeness
year_built	Keep	Absolute value; building_age is dropped as redundant
owner_occupied	Keep	Categorical, low cost; might add small lift
assessed_value	Model A only	Used in Assessment-aware AVM; excluded from Fundamentals-only AVM
latitude, longitude, zip_code	Drop	Random — see Findings 4 and 5
price_per_sqft	Drop	Target leakage — derived from last_sale_price / sqft
annual_tax, building_age	Drop	Redundant with assessed_value and year_built respectively
last_sale_date, days_since_sale, sale_year, tax_year	Drop	Temporal noise — see Finding 7
id, full_address, street_address, unit, owner_name, assessor_id	Drop	Identifiers and high-cardinality strings

Realistic accuracy expectations

The originally proposed accuracy targets (under 15% MAPE, stretch 10%) were set assuming real-data feature behaviour. Given the findings, expectations should be updated as follows:

Model	Expected R ²	Expected MAPE
Naive baseline (overall mean)	0.00	~50%
State-mean baseline	~0.57	~30–35%
Model B: Fundamentals-only AVM	0.65–0.75	25–35%
Model A: Assessment-aware AVM	0.95–0.99	5–10%

Model A's strong performance reflects the structure of the data — a near-perfect proportional relationship between `assessed_value` and `sale_price` — rather than meaningful learned valuation insight. Model B is the more honest test of what the dataset's other features can teach a valuation model.

Recommendations

For Phase 1

- **Proceed with the two-model approach.** Build Model A (Assessment-aware) as a benchmark and Model B (Fundamentals-only) as the substantive deliverable. Present both honestly in the final writeup, including their trade-offs.
- **Implement the cleaning pipeline as a reproducible script.** Document each cleaning decision (BOM stripping, low-end price filter, column drops) so the pipeline can be re-run on any future data refresh.
- **Use target encoding for state and city.** Fit on the training fold only and apply to test. This captures the dominant location signal compactly.
- **Apply log transformation to `last_sale_price` for training.** The target is right-skewed; `log1p / expm1` handles this and produces more stable training.

For Phase 2

- **Onboard a real listings or transaction dataset.** The current dataset cannot support marketplace, property management, or mortgage services as specified in the original brief.
- **Source verifiable physical attributes.** A dataset where `sqft`, `bedrooms`, `bathrooms`, and property type carry their normal real-world signal would substantially improve the AVM beyond what Phase 1 can achieve.
- **Source real geographic identifiers.** Verifiable zip codes, latitude, and longitude open up spatial features (distance to amenities, school district lookups, comparables search), all of which are unavailable today.
- **Add temporal data on listings.** Days-on-market, listing date, and listing-to-sale conversion are needed for any market-trend or demand-side analytics that go beyond the current historical-snapshot view.

Appendix: Methodology

The data quality assessment was conducted using a structured 12-step procedure:

- **Setup and reload.** CSV loaded with explicit delimiter, encoding, dtype, and date-parsing parameters.
- **Structural checks.** Shape, dtypes, duplicate rows, primary key uniqueness, ID-pair relationships.
- **Missingness analysis.** Per-column NaN counts and percentages, with cross-tabulation against potential explanatory columns to distinguish structural from random missingness.
- **Numeric range plausibility.** Each numeric column checked against domain-appropriate plausibility bounds (e.g., bedrooms 0–15, latitude 18–72, year_built 1800–2026).
- **Categorical sanity.** Value counts for property_type, state, owner_occupied; check for typo variants; validation of US state codes; cities-in-multiple-states detection.
- **Cross-field consistency.** Verification that derived columns match their source columns (price_per_sqft, building_age + year_built, days_since_sale).
- **Target distribution.** Histograms (raw and log), boxplot, summary statistics, skewness.
- **Temporal patterns.** Median price and sale count by year to detect appreciation trends or assignment irregularities.
- **Feature-target relationships.** Scatter plots and boxplots of each feature against last_sale_price to assess signal strength.
- **Correlation and leakage analysis.** Pearson correlations of all numeric features with the target; detection of features correlating > 0.95 (leakage flags).
- **Geographic patterns.** Median price by state and city; coordinate-state matching against known US bounding boxes.
- **Data-quality summary.** Consolidated findings, decisions, and modelling-feature plan.

All code and intermediate outputs are preserved in the project notebook for reproducibility.